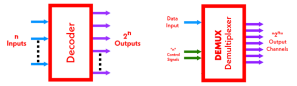
Architectures logicielles et matérielles

Cours 7 : Circuits combinatoires

Laurence Pierre

Introduction

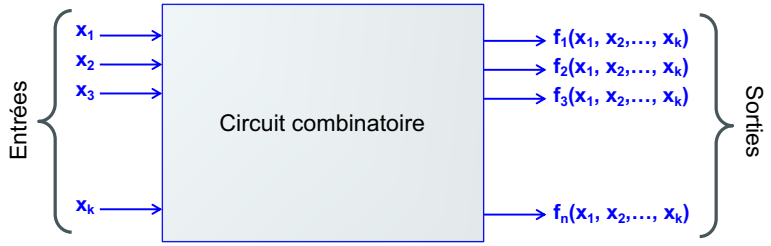
- Nous passons à la **partie Conception matérielle** de l'UE
  - Commençons par la conception des **circuits combinatoires**, qui sera illustrée, entre autres, par le circuit *décodeur* (et le *démultiplexeur*) évoqué précédemment



  - Puis la conception de **circuits séquentiels de contrôle**
  - Pour finir par la conception de **circuits séquentiels** contenant **contrôle et calculs**, dédiés à une application particulière
  - En conclusion, nous fournirons quelques éléments d'ouverture vers des concepts plus avancés :
    - langages et outils pour la conception matérielle
    - notions de sécurité (attaques matérielles)

Conception de circuits combinatoires

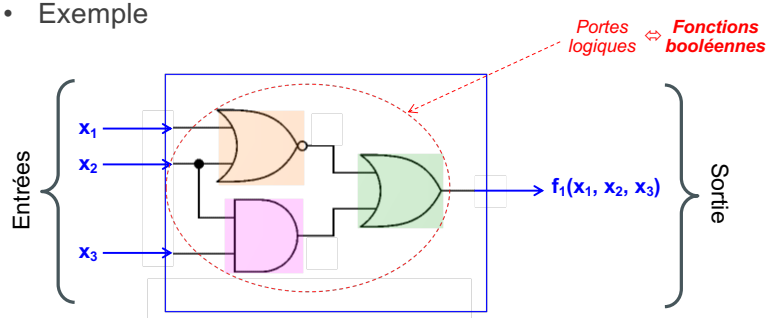
- Les circuits combinatoires correspondent à la mise en œuvre de **fonctions booléennes**



  - $x_1, x_2, \ldots x_k$: variables logiques (vrai/faux, 1/0)
  - $f_1(x_1,\ldots x_k)$, $f_2(x_1,\ldots x_k)$,…, $f_n(x_1,\ldots x_k)$: fonctions de ces variables
- Objet de ce cours : comment génère-t-on et synthétise-t-on ces fonctions ?

Conception de circuits combinatoires

- Exemple



  - $f_1(x_1,x_2,x_3)$ = not $(x_1$ or $x_2)$ or $(x_2$ and $x_3)$

Fonctions booléennes

Rappels sur les opérateurs booléens

- Les principaux opérateurs de l'algèbre de Boole sont
  - La disjonction (or) : (x | y) ou (x ∨ y) ou (x + y)
  - La conjonction (and) : (x & y) ou (x ∧ y) ou (x . y)
  - La négation (not) : (! x) ou (¬ x) ou ($\overline{x}$)

a	b	a or b
0	0	0
0	1	1
1	0	1
1	1	1

a	b	a and b
0	0	0
0	1	0
1	0	0
1	1	1

a	not a
0	1
1	0

Rappels sur les opérateurs booléens

- D'autres opérateurs peuvent être construits à partir de ces opérateurs de base :
  - nand (non et) :
    - a nand b = not (a and b)
  - nor (non ou) :
    - a nor b = not (a or b)
  - xor (ou exclusif) :
    - a xor b = (a and not b) or (not a and b)
  - implication :
    - a => b = not a or b
  - equivalence :
    - a <=> b = (a => b) and (b => a) = (not a or b) and (not b or a)

Rappels sur les opérateurs booléens

- D'autres opérateurs peuvent être construits à partir de ces opérateurs de base :
  - nand (non et) :
    - a nand b = not (a and b)
  - nor (non ou) :
    - a nor b = not (a or b)
  - xor (ou exclusif) :
    - a xor b = (a and not b) or (not a and b)
  - implication :
    - a => b = not a or b
  - equivalence :
    - a <=> b = (a => b) and (b => a) = (not a or b) and (not b or a)

a	b	a nand b
0	0	?
0	1	
1	0	
1	1	

a	b	a nor b
0	0	?
0	1	
1	0	
1	1	

a	b	a xor b
0	0	?
0	1	
1	0	
1	1	

Rappels sur les opérateurs booléens

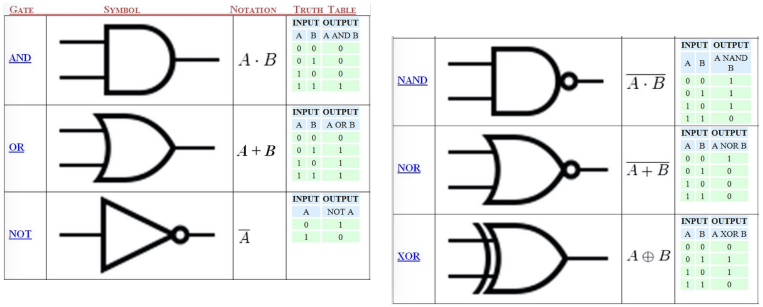
- D'autres opérateurs peuvent être construits à partir de ces opérateurs de base :
  - nand (non et) :
    - a nand b = not (a and b)
  - nor (non ou) :
    - a nor b = not (a or b)
  - xor (ou exclusif) :
    - a xor b = (a and not b) or (not
  - implication :
    - a => b = not a or b
  - equivalence :
    - a <=> b = (a => b) and (b => a) = (not a or b) and (not b or a)

a	b	a => b
0	0	?
0	1	
1	0	
1	1	

a	b	a <=> b
0	0	?
0	1	
1	0	
1	1	

Portes logiques

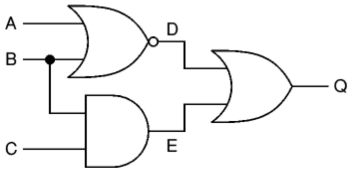
- Chaque opérateur booléen se traduit par une "porte" logique :



http://electr-engr-world.blogspot.com/2016/05/logic-gates-symbol-notation-truth-table_25.html

Portes logiques

- Un assemblage de portes logiques réalise une fonction booléenne
  - Par composition fonctionnelle, la sortie (primaire) s'exprime en fonction des entrées (primaires)
- Exemple :



D = NOT (A OR B)
E = B AND C
Q = D OR E = (NOT (A OR B)) OR (B AND C)

https://www.dse.nl/~boncoeur/logic_gates.html

Conception et synthèse logique

- A partir de la description de la fonctionnalité souhaitée, comment **concevoir le composant matériel** (combinatoire, ici) correspondant ?



- Par exemple

Le composant matériel voulu doit prendre en entrées 8 bits de données $D_0 \ldots D_7$ et un entier N codé sur 3 bits, et transmettre sur son unique sortie la donnée D_N dont le numéro est codé par N

Conception et synthèse logique

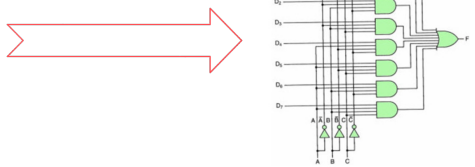
- A partir de la description de la fonctionnalité souhaitée, comment **concevoir le composant matériel** (combinatoire, ici) correspondant ?



- Par exemple

Le composant matériel voulu doit prendre en entrées 8 bits de données $D_0 \ldots D_7$ et un entier N codé sur 3 bits, et transmettre sur son unique sortie la donnée D_N dont le numéro est codé par N

Comment obtenir un circuit à partir d'un tel énoncé ??



Conception et synthèse logique

- Illustrons les principes sur un exemple
  - Exemple du calcul de la **somme** de 2 entiers non signés
    - Principe de l'addition en décimal…

```
  1 0 0 1 1 0
+ 3 4 5 8 9 0
  8 1 2 3 5 6
-----------
1 1 5 8 2 4 6
```

    - Et même principe en binaire (addition à retenue propagée)

```
  1 1 1 1 1 0 0 0
+ 1 1 0 1 1 1 0 0
  0 0 1 1 1 0 1 0
---------------
1 0 0 0 1 0 1 1 0
```


Conception et synthèse logique

- Illustrons les principes sur un exemple
  - Exemple du calcul de la **somme** de 2 entiers non signés
    - Principe de l'addition en décimal…

```
  1 0 0 1 1 0
+ 3 4 5 8 9 0
  8 1 2 3 5 6
-----------
1 1 5 8 2 4 6
```

    - Et même principe en binaire (addition à retenue propagée)

```
  1 1 1 1 1 0 0 0
+ 1 1 0 1 1 1 0 0
  0 0 1 1 1 0 1 0
---------------
1 0 0 0 1 0 1 1 0
```

Faire la somme de 3 bits et produire le résultat sur 2 bits (le bit de poids fort est la retenue)

Conception et synthèse logique

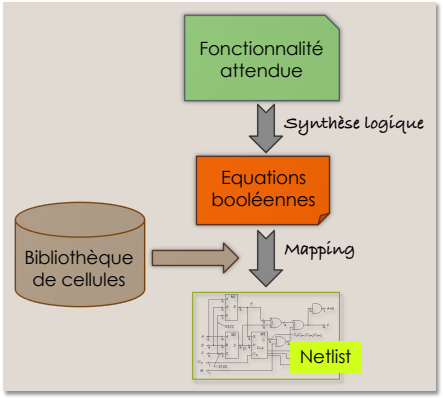
- Illustrons les principes sur un exemple
  - Exemple du calcul de la **somme** de 2 entiers non signés
    - Comment faire cette "**addition 1 bit**" (somme de 2 bits et d'un bit de retenue entrante) ?

A	B	C	R_1	R_0
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Et comment utiliser cette spécification pour produire un composant matériel ? => **synthèse logique**

Synthèse logique

- Il s'agit de transformer une spécification de la fonction voulue en un assemblage de portes logiques (netlist)



Une équation booléenne par sortie du bloc combinatoire

Synthèse logique

- Il s'agit de transformer une spécification de la fonction voulue en un assemblage de portes logiques (netlist)
- Principe pour l'**obtention des équations** :
  - Etape 1 : formalisation de la fonction voulue pour chaque sortie, par exemple sous la forme d'une table de vérité (1)
  - Etape 2 : on en dérive une expression algébrique, soit sous forme d'une *somme de produits* ("minterms") soit sous forme d'un *produit de sommes* ("maxterms") (2)
    - Pour obtenir une somme de produits : on considère les lignes de la table de vérité pour lesquelles *la sortie vaut 1*. Dans chaque produit, les *variables d'entrée qui ont la valeur 1 sont prises telles quelles et les autres sont complémentées*
  - Etape 3 : il reste à simplifier l'expression obtenue (3)

Synthèse logique

- Continuons l'illustration sur l'exemple…
  - Exemple du calcul de la **somme** de 2 entiers non signés
    - Calcul pour 1 bit des deux entiers

(1)

A	B	C	R_0
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

A	B	C	R_1
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

(2)

R_0 = (not a).(not b).c + (not a).b.(not c) + a.(not b).(not c) + a.b.c

R_1 = (not a).b.c + a.(not b).c + a.b.(not c) + a.b.c

- Mais il faut aussi simplifier ces expressions => phase (3)

Synthèse logique

- Pour simplifier ces expressions (phase (3) slide 19), il faut avoir recours aux lois de l'algèbre de Boole*
  - Lois de De Morgan (de Augustus De Morgan)
    - not (a . b) = (not a) + (not b)
    - not (a + b) = (not a) . (not b)
  - En effet

a	b	not (a . b)	(not a) + (not b)	not (a + b)	(not a) . (not b)
0	0	1	1	1	1
0	1	1	1	0	0
1	0	1	1	0	0
1	1	0	0	0	0

* La simplification peut aussi être aisée par l'utilisation de tables de Karnaugh en alternative aux tables de vérité (voir compléments)

Synthèse logique

- Pour simplifier ces expressions (phase (3) slide 19), il faut avoir recours aux lois de l'algèbre de Boole
  - Idempotence :
    - a . a = a a + a = a
  - Lois d'identité :
    - 1 . a = a 0 + a = a
  - Lois de nullité :
    - 0 . a = 0 1 + a = 1
  - Lois d'inversion :
    - a . not(a) = 0 a + not(a) = 1

Synthèse logique

- Pour simplifier ces expressions (phase 3 slide 19), il faut avoir recours aux lois de l'algèbre de Boole
  - Commutativité :
    - a . b = b . a a + b = b + a
  - Associativité :
    - (a.b).c = a.(b.c) (a+b)+c = a+(b+c)
  - Distributivité :
    - a+(b.c) = (a+b).(a+c) a.(b+c) = (a.b)+(a.c)
  - Lois d'absorption :
    - a. (a + b) = a a + (a . b) = a

Synthèse logique

- Toutes ces lois permettent de transformer/simplifier les expressions booléennes
- Exemple : simplifier (a + b) . (a + not b) . not(a . (not c))

?

Synthèse logique

- Continuons l'illustration sur l'exemple…
  - Exemple du calcul de la **somme** de 2 entiers non signés
    - Les **équations** produites à partir des tables de vérité étaient

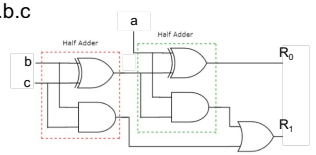
R_0 = (not a).(not b).c + (not a).b.(not c) + a.(not b).(not c) + a.b.c

R_1 = (not a).b.c + a.(not b).c + a.b.(not c) + a.b.c

    - Elles se **simplifient** comme suit

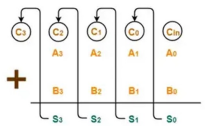
R_0 = (not a).(not b).c + (not a).b.(not c) + a.(not b).(not c) + a.b.c
= (not a).((not b).c + b.(not c)) + a.((not b).(not c) + b.c)
= (not a).(b xor c) + a.(b equ c)
= (not a).(b xor c) + a.(not(b xor c)) = **a xor (b xor c)**

R_1 = (not a).b.c + a.(not b).c + a.b.(not c) + a.b.c
= b.c.(not a + a) + a.(not b).c + a.b.(not c)
= b.c + a.((not b).c + b.(not c))
= **b.c + a.(b xor c)**

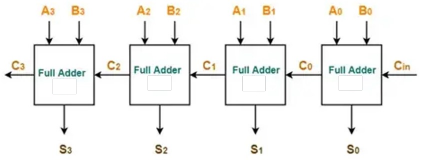


Synthèse logique

- Continuons l'illustration sur l'exemple…
  - Exemple du calcul de la somme de 2 entiers non signés
    - Le composant obtenu est un "**additionneur 1 bit**" (full adder)
    - Il reste à assembler ces full adders pour sommer en propageant la retenue…



    - On obtient un additionneur à retenue propagée (ripple carry adder)



https://siliconvlsi.com/ripple-carry-adder/

Synthèse logique

- Autre exemple : l'entrée est un vecteur de 3 bits $x_2\ x_1\ x_0$, sortie à 1 si $x_2\ x_1\ x_0$ représente un palindrome

Synthèse logique

- Autre exemple : l'entrée est un vecteur de 3 bits $x_2\ x_1\ x_0$, sortie à 1 si $x_2\ x_1\ x_0$ représente un palindrome

1

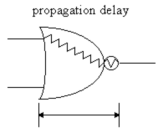
x_2	x_1	x_0	S
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

- S = ?

En **exercice** à préparer : appliquer les étapes 2 et 3 du slide 19, et produire ainsi l'équation de S

Notion de délai

- Jusqu'ici, nous avons considéré un circuit combinatoire comme la représentation d'une fonction **booléenne**
- Une vision plus concrète des portes logiques nous conduit à prendre en compte une **notion de temps**
- Les portes réalisent des opérateurs booléens, mais sont des composants électroniques qui subissent un *temps de propagation* : temps écoulé entre la présence de valeurs stables sur les entrées de la porte et la présence d'une valeur stable sur sa sortie

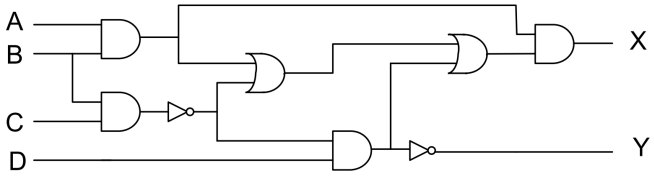


Notion de délai

- Il faut avoir conscience du *délai de stabilisation* d'un bloc combinatoire : temps écoulé entre un changement sur les entrées et la stabilisation des sorties en conséquence (qui peuvent passer par des états transitoires dans cet intervalle)
- Une notion importante est le *chemin critique* :
  - À chaque chemin d'une entrée vers une sortie correspond un délai (somme des temps de propagation des portes sur ce chemin)
  - Le chemin critique est le **chemin dont le délai est maximal** (pas forcément unique)

Notion de délai

- Chemin critique - Exemple :
  - En supposant que le temps de propagation des portes NOT du circuit ci-dessous est de 1 ns, et celui des autres portes est 2 ns



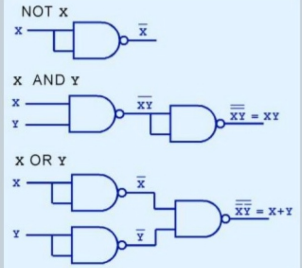
  - **Quels chemins** et **délais** associés ?
    Quel est le **chemin critique**, quel est son délai ?

Compléments

I.Transformation en NAND/NOR

- L'obtention des équations finales se fait fréquemment en visant l'utilisation de **portes NAND ou NOR**, notamment dans ce contexte de synthèse ASIC (Application-Specific Integrated Circuit)

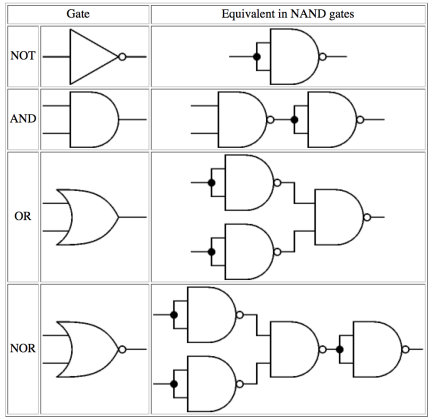
NAND and NOR are known as *universal gates* because they are inexpensive to manufacture and any Boolean function can be constructed using only NAND or only NOR gates.



http://fr.slideshare.net/KamalAcharya/introduction-to-digital-logic

I.Transformation en NAND

- Portes équivalentes



not a = a nand a

a and b = not (a nand b)

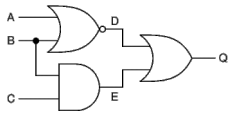
a or b = not not (a or b)
= not ((not a) and (not b))
= (not a) nand (not b)

a nor b = not (a or b)
= (not a) and (not b)
= not ((not a) nand (not b))

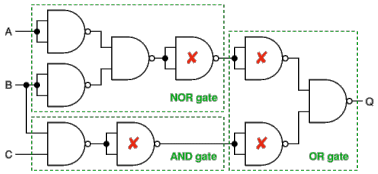
https://www.dse.nl/~boncoeur/logic_gates.html

I.Transformation en NAND

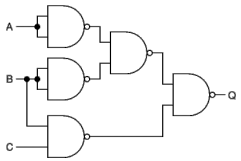
- Exemple : q = not (a + b) + b.c



  - q = not (a + b) + b.c
    = not ((not a) nand (not b)) + not (b nand c)
    = not (not ((not a) nand (not b))) nand not (not (b nand c))



= ((not a) nand (not b)) nand (b nand c)



a + b = (not a) nand (not b)
a . b = not (a nand b)

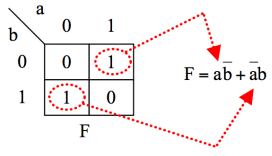
https://www.dse.nl/~boncoeur/logic_gates.html

II.Tables de Karnaugh

- Alternative à la représentation d'une fonction booléenne sous forme de table de vérité (aisée pour les fonctions de 2 à 4 variables)
- Peut permettre de faciliter le processus de simplification de l'expression booléenne
- Cas à 2 variables. Exemple : F(a,b) = a xor b

a	b	F
0	0	0
0	1	1
1	0	1
1	1	0

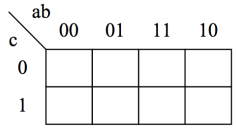
Equivalence à la table de vérité

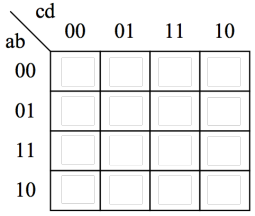


http://dado59.free.fr/www2/cours/automatique/karnaugh.pdf

II.Tables de Karnaugh

- Cas de fonctions à 3 et 4 variables :

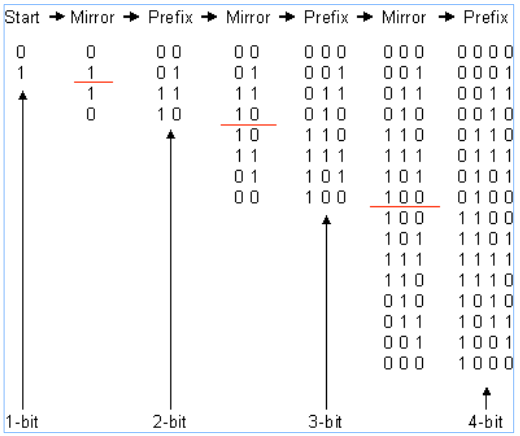




- Noter que les codes utilisés sur les colonnes ou lignes consécutives sont tels qu'ils ne diffèrent que d'un seul bit : il s'agit du *code de Gray*

Note. Code de Gray

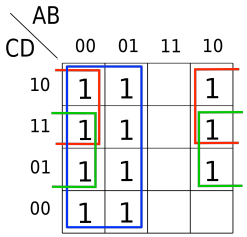
Décimal	Binaire	Gray
0	000	000
1	001	001
2	010	011
3	011	010
4	100	110
5	101	111
6	110	101
7	111	100



https://www.edn.com/logic-101-part-4-gray-codes/

II. Tables de Karnaugh

- Exemple :



- s = (not a) + (not b).c + (not b).d

a	b	c	d	s
0	0	0	0	1
0	0	0	1	1
0	0	1	0	1
0	0	1	1	1
0	1	0	0	1
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	0
1	0	0	1	1
1	0	1	0	1
1	0	1	1	1
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0

https://fr.wikibooks.org/wiki/%C3%89lectronique_num%C3%A9rique_:_logique/Simplification_et_implantation_de_formes_disjonctives